Evaluating Association between Binary Response Variables using Test of Independence

Based on survey data

Department of Statistics

Pritika Saha

(Registration no: 23214220006)

Roll no: STAT006

Semester-5

Class- UG-3

Under the supervision of

Dr. Sumanta Adhya

Assistant Professor

West Bengal State University

Summer Internship

(Paper Code: STAT341SI01)

Acknowledgement

I would like to express my sincere gratitude to Dr. Sumanta Adhya sir, Assistant Professor at West Bengal State University, under whose valuable guidance and encouragement I was able to complete this report successfully.I am also thankful to my professors from the Department of Statistics, Presidency University, for their constant support and guidance.

Contents

1 Introduction **3**

2 Dataset Description : **4**

2.0.1 Contingency Table of Voting and Binary income status . 4

3 Based on Finite Population **5**

4 Test Based on Design-Sample **6**

4.1 Justification for Estimating Population Probabilities Using Sample Data 7

4.2 Simulation Study for χ^2-test based on Sample 7

5 Test for Independence Using Auxiliary Information **9**

5.1 Simulation 11

5.1.1 Comparison of Sample(T_s) and model-based(T_m) p-values 13

6 Test under Stratified Sampling **14**

6.1 Simulation: 15

6.2 Test for Independence Using complete Auxiliary Information under Statified Sampling 17

6.3 Simulation 17

7 Population Generation using the Bivariate Probit Model **19**

8 Conclusion **23**

9 References **24**

10 Appendix **24**

1 Introduction

This report investigates the association between voting behavior and income status, both measured as binary variables. we are provided with a finite population dataset, we assume that it represents a realization from an underlying infinite population. Under this perspective, drawing a sample from the finite population is treated as a way of approximating inference about the infinite population.We begin with the full population and apply the chi-square test of independence to assess whether voting and income are related. Since in practice only samples are observed, we replicate this analysis under Simple Random Sampling Without Replacement (SRSWOR) and stratified sampling, comparing sample-based results with the population test.

To strengthen inference, we also use a bivariate probit model that jointly models voting and income while conditioning on covariates. This allows construction of a model-based test. We have to shown that stratified sampling yields more stable estimates than SRSWOR, and the model-based approach provides valid population-level conclusions with reduced variability, even when only sample data are available.And also shown that the model based test follows chi-square distribution with df=1 under independence.

2 Dataset Description :

We begin our analysis using the voteincome dataset available in the ZeligChoice package in R. This dataset contains survey responses related to voting behavior and income levels, among other variables. Our focus is on examining the relationship between whether a respondent voted and their level of income.

The first few rows of the dataset look like this:

```
  state year vote income education age female
1    AR 2000    1      9         2  73      0
2    AR 2000    1     11         2  24      0
3    AR 2000    0     12         2  24      1
4    AR 2000    1     16         4  40      0
5    AR 2000    1     10         4  85      1
6    AR 2000    1     12         3  78      1
```

To simplify the analysis, we transform the continuous income variable into a binary variable called binary_income_status. This new variable indicates whether an individual's income is above the median income:

- binary_income_status =1, if income > median income
- binary_income_status =0 , otherwise

The reason behind this step is to group respondents into two broad categories — high and low income to explore whether income level is associated with voting behavior.

The few rows of the new dataset look like this:

```
  vote binary_income_status education age female
1    1                    0         2  73      0
2    1                    0         2  24      0
3    0                    0         2  24      1
4    1                    1         4  40      0
5    1                    0         4  85      1
6    1                    0         3  78      1
```

2.0.1 Contingency Table of Voting and Binary income status

We create a 2×2 contingency table to examine the joint distribution of the binary variables vote (1 if voted, 0 if not) and binary_income_status:

```
Frequency Table(f_ij):
    Income
Vote   0   1
   0 149  68
   1 616 667
```

This table provides the observed frequencies of individuals who:

- Voted and had high income
- Voted and had low income
- Did not vote and had high income
- Did not vote and had low income

We also create a proportion table($p_{ij} = \frac{f_{ij}}{n}$) :

```
Proportion Table(p_ij):
    Income
Vote     0     1
   0 0.099 0.045
   1 0.411 0.445
```

3 Based on Finite Population

To formally test whether voting behavior and income level are statistically associated, we use the Chi-square Test of Independence. This test evaluates the null hypothesis:

$$H_0 : Voting\ and\ Income\ are\ independent$$

The test statistic is calculated as:

$$\chi^2 = \sum_{i=1}^{2}\sum_{j=1}^{2}\frac{(f_{ij} - E_{ij})^2}{E_{ij}}, df = (2-1)*(2-1) = 1$$

Where :

- f_{ij} = observed frequency in cell (i,j)
- $E_{ij} = n\hat{p}_{1.} * \hat{p}_{.1}$= expected frequency under independence

If the calculated chi-square value is large and the corresponding p-value is small (typically < 0.05), we reject the null hypothesis, suggesting there is a significant association between income and voting behavior.

1. Test statistic and p-value :

We apply the chi-square test on the 2×2 contingency table constructed from the voteincome dataset

```
Chi-square statistic = 31.675
 p-value: 2.7819e-08
```

Since the p-value ($2.7819e^{-8}$) is less than 0.05, we reject the null hypothesis of independence. This indicates that there is a statistically significant association between income status and voting behavior in the sample.

4 Test Based on Design-Sample

Although the chi-square test applied to the full voteincome dataset provides insight into the association between voting and income, in real-world studies, we often have access to only sample data rather than the full population.To replicate such practical scenarios and study the sampling behavior of the chi-square statistic, we draw a random sample from the dataset.

we draw a sample from the voteincome dataset using Simple Random Sampling Without Replacement (SRSWOR).

We select a single sample of size 200 from the population without replacement to ensure that no individual is selected more than once. From this sample.

We now apply the same procedure to the sample data, just as we did with the full dataset.

- Contingency table :

From the selected sample, we construct the 2×2 contingency table of vote and binary_income_status

```
Frequency Table(f_ij):
    Income
Vote  0  1
   0 21  8
   1 78 93
```

Proportion table:

```
Proportion Table(p_ij):
    Income
Vote     0     1
   0 0.105 0.040
   1 0.390 0.465
```

- Test statistic and p-value :

```
Chi-square statistic = 7.1241
p-value = 0.007606
```

Since the p-value (0.007606) is less than the conventional significance level of 0.05, we reject the null hypothesis of independence in the sample as well.

4.1 Justification for Estimating Population Probabilities Using Sample Data

We denote each unit in the population by $\{y_i = \begin{pmatrix} y_{1i} \\ y_{2i} \end{pmatrix}, x_i, R_i\}$, i = 1, 2, 3,...,N , where N= population size

where $R_i$is a sample indicator defined as $R_i = \begin{cases} 1 & if \quad i \quad \epsilon \quad sample \\ 0 & otherwise \end{cases}$

To compute population level probabilities using sample data and covariates we rely on the assumption that sample inclusion is ignorable given covariates .we assume:

$$p(R_i = 1|y_{1i} = 1, y_{2i} = 1, x_i = x_i) = p(R_i = 1|x_i = x_i)$$

Then using Bayes' rule , we derive
$p(y_{1i} = 1, y_{2i} = 1|x_i, R_i = 1)$
$= \frac{p(y_{1i}=1,y_{2i}=1,x_i=x_i,R_i=1)}{p(R_i=1,x_i=x_i)}$
$= \frac{p(R_i=1|y_{1i}=1,y_{2i}=1,x_i=x_i)*p(y_{1i}=1,y_{2i}=1|x=x_i)*p(x_i=x_i)}{p(R_i=1|x_i=x_i)*p(x_i=x_i)}$
$= p(y_{1i} = 1, y_{2i} = 1|x = x_i)$, for all i ϵ sample
so we can write $p(y_{1i} = 1, y_{2i} = 1|x = x_i) = (y_{1i}, y_{2i}, x_i) \mid (R_1, R_{2,}..., R_N)$, for all i ϵ sample

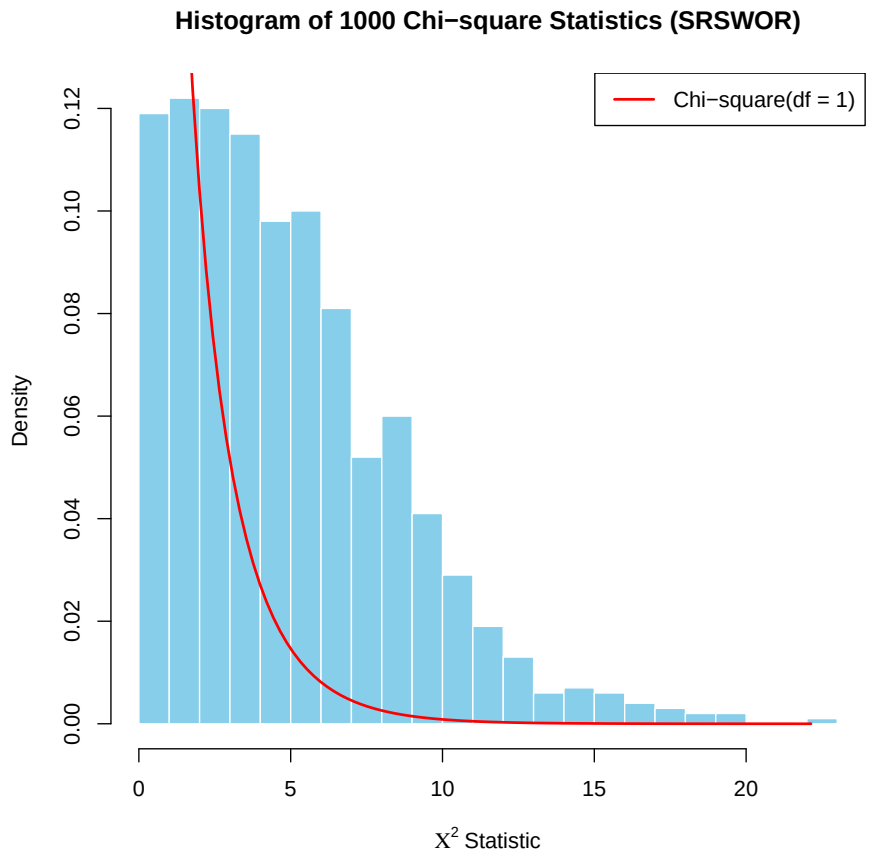
This implies that, conditional on covariates, the joint distribution of (y_{1i}, y_{2i}) in the sample equals that in the population.

4.2 Simulation Study for χ^2-test based on Sample

To better understand the variability and distribution of the chi-square statistic under repeated sampling, we perform a simulation by repeating the above sampling process 1000 times.Then for each:

- Draw a Simple Random Sample Without Replacement (SRSWOR) of size 200 from the full voteincome dataset.
- Construct a 2×2 contingency table of vote and binary_income_status.
- Perform a Chi-square Test of Independence.
- Record the resulting chi-square statistic and p-value from each sample.

a. We now visualize the empirical distribution of the 1000 chi-square statistics and compare it with the theoretical chi-square distribution with 1 degree of freedom:



The histogram shows the distribution of chi-square statistics from 1000 SRSWOR samples, overlaid with the theoretical chi-square curve (df = 1). The empirical distribution closely follows the theoretical curve, especially in the lower range, confirming that the test statistic approximately follows the chi-square distribution under repeated sampling. This supports the validity of the chi-square test in assessing independence between voting and income.

b. To estimate how often the null hypothesis of independence is rejected under repeated sampling, we examine the proportion of samples with a p-value less than 0.05 at the 5% level . It gives us

```
Number of samples with p-value < 0.05: 538 out of 1000
Percentage of reject the null hypothesis: 53.8 %
```

Out of 1000 samples, 538 (53.8%) had p-values below 0.05, indicating rejection of the null hypothesis in over half the cases. This suggests that the chi-square test can detect the association between income and voting in many samples, but not all — likely due to sampling variability in smaller sample sizes.

5 Test for Independence Using Auxiliary Information

While our previous simulation used only observed sample frequencies to test independence, it did not incorporate auxiliary population information beyond the sample. In many real-world scenarios, we may want to combine what we know from a sample with model-based expectations derived from a fitted model over the rest of the population.

Formally, we consider the testing problem: $T(p_{11}, p_{10}, p_{01}, p_{00})$

Where (y_1, y_2) are binary outcomes and x are covariates. Under a finite population, randomness arises due to the sampling design. We use the observed sample together with model-based estimates to obtain expected probabilities, and then construct a plug-in test statistic: $\hat{T}(\hat{p_{11}}, \hat{p_{10}}, \hat{p_{01}}, \hat{p_{00}})$

The method estimates marginal probabilities over the entire population by combining:

- Direct observations from the sample, and
- Model-based expectations from the non-sample, using a fitted Bivariate probit model.

To estimate the overall voting rate $(\hat{p_{1.}})$ and income rate $(\hat{p_{.1}})$ „ we use the following formula:

$$\hat{p}_{1.} = \underbrace{\frac{1}{N}[\sum_{i=0}^{n} I(y_{1i} = 1)}_{Known} + \underbrace{\sum_{i=n+1}^{N} \hat{E}(I(y_{1i} = 1)|x_i)]}_{Unknown} = \frac{1}{N}[\sum_{i=0}^{n} I(y_{1i} = 1) + \sum_{i=n+1}^{N} \hat{p}((y_{1i} = 1)|x_i)]$$

$$\hat{p}_{.1} = \underbrace{\frac{1}{N}[\sum_{i=0}^{n} I(y_{2i} = 1)}_{Known} + \underbrace{\sum_{i=n+1}^{N} \hat{E}(I(y_{2i} = 1)|x_i)]}_{Unknown} = \frac{1}{N}[\sum_{i=0}^{n} I(y_{2i} = 1) + \sum_{i=n+1}^{N} \hat{p}((y_{2i} = 1)|x_i)]$$

where,

- N = total population size,
- n = sample size,
- $I(.)$ = indicator function from the sample,
- $P(.|x_i)$ = predicted probability from the Bivariate probit model for the non-sample observations.

These estimated marginals are then used to compute the expected frequencies under independence, and the test statistic is calculated as:

$$Test - Statistic = \sum_{i=1}^{2}\sum_{j=1}^{2} \frac{(f_{ij} - n\hat{p}_{1.} * \hat{p}_{.1})^2}{n\hat{p}_{1.} * \hat{p}_{.1}}$$

This approach improves inference by incorporating additional structure learned from the full dataset via the Bivariate probit model (vote and high_income jointly modeled by education).

- Application of the Bivariate Probit Model :

In this study, we employ the Bivariate Probit model to jointly examine the relationship between two binary outcomes: whether an individual voted (y_{1t}) and whether their income is above the median (y_{2t}) . Since both outcomes are potentially dependent due to unobserved common factors, the bivariate probit framework provides a natural structure for modeling their joint distribution.

Define

$y_{1t} = I(U_{1t} > 0)$, $y_{2t} = I(U_{2t} > 0)$

where the latent variables U_{1t} and U_{2t} are specified as linear functions of observed covariates plus random errors:

$$U_{1t} = \beta_0 + \beta x_t + \varepsilon_{1t}$$

$$U_{2t} = \gamma_0 + \gamma x_t + \varepsilon_{2t}$$

In our case we have chosen education as covariate so, x_t =education

where $\varepsilon_{1t}$and ε_{2t} are the error terms . The error terms following a bivariate standard normal distribution:

$$\begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right)$$

this specification allows two binary outcome to be dependent through shared latent factors (captured by ρ) while simultaneously adjustion for the effect of education.

This specification implies the following marginal and joint probabilities:

$$P[y_{1t} = 1] = P[U_{1t} > 0] = \phi(\beta x_t)....(i)$$

$$P[y_{2t} = 1] = P[U_{1t} > 0] = \phi(\gamma x_t)....(ii)$$

$$P[y_{1t} = 1, y_{2t} = 1] = P[U_{1t} > 0, U_{2t} > 0] = F(\beta x_t, \gamma x_t, \rho)....(iii)$$

where ϕ is the distribution function of a standard normal variable, and F is the bivariate distribution function of standard normal variables with correlation coefficient ρ.

To estimate the parameters of this model , we utilize the zelig package in R, specifically the bprobit as the argument. This allows joint modeling of two binary outcomes- vote and binary_income_status as function of covariate education.

we will use the estimates of regression coefficients and correlation parameter to predict the joint probabilities of the response variables for the non sampled units ($\sum_{i=n+1}^{N} \hat{p}((y_{2i} = 1)|x_i)$)

In the superpopulation framework, the observed finite population is regarded as one realization from an underlying probability model. Let $Y_i$denote the response for the i^{th}unit and let $f(y_i|\underset{\sim}{\theta})$ denote the probability mass\density function governed by parameter vector $\underset{\sim}{\theta}$.

For a sample of size n , the likelihood function is defined as : $L(\underset{\sim}{\theta}) = \prod_{i=1}^{n} f(y_i|\underset{\sim}{\theta})$

and the corresponding log-likelihood is : $l(\underset{\sim}{\theta})= \sum_{i=1}^{n} f(y_i|\underset{\sim}{\theta})$

Now, the parameter $\underset{\sim}{\theta}$ are obtained by maximizing the corresponding log-likelihood . These estimates are then used to predict the joint probabilities for the non-sampled units, which ,when combined with the observed sample proportions , yield the model based estimators for the population cell probabilities .

After fitting the model and computing the Test-statistic, we obtained:

```
How to cite this model in Zelig:
  Thomas W. Yee. 2007.
  bprobit: Bivariate Probit Regression for Dichotomous Dependent Variables
  in Christine Choirat, Christopher Gandrud, James Honaker, Kosuke Imai, Gary King, and Oliv
  "Zelig: Everyone's Statistical Software," https://zeligproject.org/
```

Test-statistic and p-value:

```
Test-statistic: 8.7407
p-value: 0.003112
```

Since the p-value(0.003112) is far below 0.05, we again reject the null hypothesis of independence. Unlike the standard chi-square test, this result accounts for the population-wide relationship modeled between voting and income, conditional on education. This suggests a robust association between income status and voting behavior in the population beyond the observed sample.

5.1 Simulation

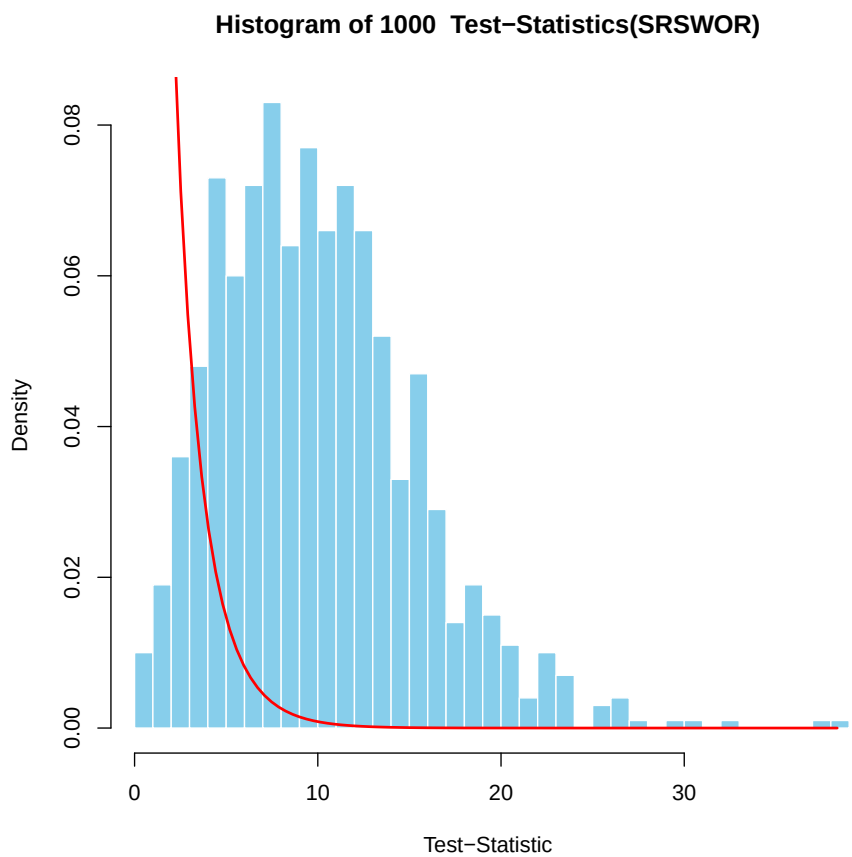
To better understand the variability and distribution of the test statistic under repeated sampling, we conduct a Monte Carlo simulation. Specifically, we draw 1000 simple random samples without replacement (SRSWOR) of size n=200 from the full voteincome population.

For each sample, we:

- Construct a 2×2 contingency table of vote vs binary_income_status.

- Estimate the model parameters using a Bivariate probit fit on the full population.
- Compute the model-based expected joint probabilities from the fitted model.
- Record the resulting Test-statistic and p-value from each sample.

  a. The resulting distribution of 1000 Test-statistic values is visualized in the histogram below, with the theoretical chi-square density overlaid in red for comparison. This comparison demonstrates wheather the Test-statistic approximately follows the expected χ^2_1distribution under the null.



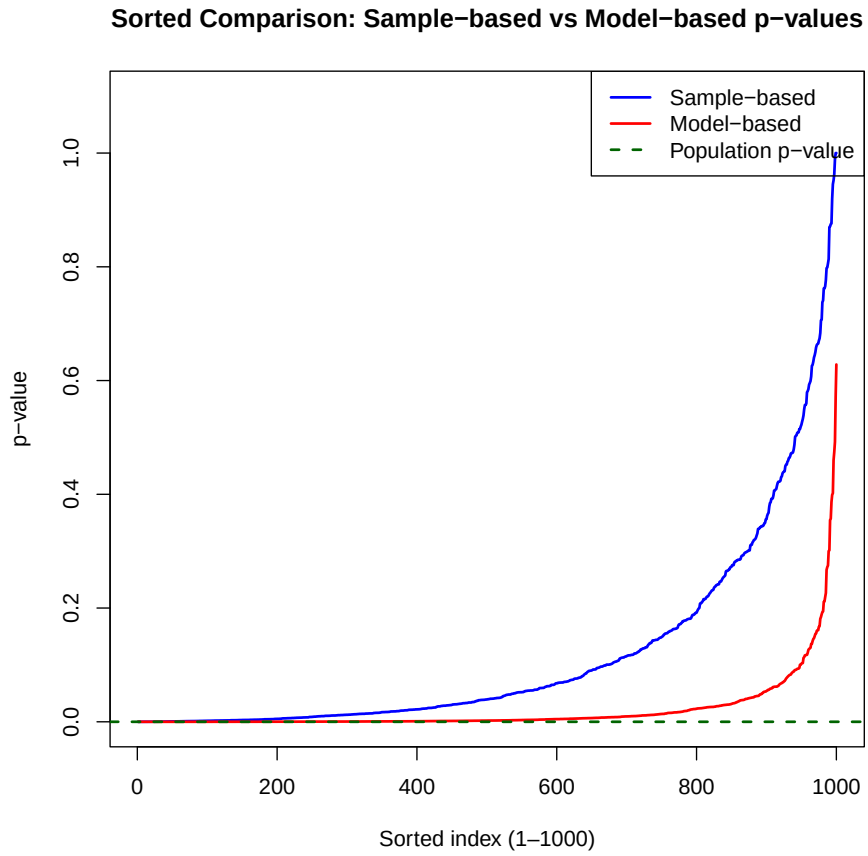
The histogram displays that the test-statistic, under the null hypothesis and with finite sample size, may not follow the theoretical chi-square distribution precisely.Specifically, the empirical distribution is right-skewed but has a peak that is shifted to the right and heavier tails compared to the theoretical curve.

b. To assess the reliability of this test evaluated how often the test rejects the null hypothesis at the 5% significance level. For each of the 1000 samples, a p-value was calculated, and we counted how many of them were less than 0.05. This yields an empirical rejection rate—i.e., the proportion of times the test indicates a significant association between voted and binary_income_status. If the test is well-calibrated under the null hypothesis, this percentage should be close to 5%. The result is

```
Number of p-values < 0.05: 893 out of 1000
Percentage of reject the null hypothesis: 89.3 %
```

Out of 1000 samples, 893 p-values were less than 0.05, leading to an 89.3% rejection rate of the null hypothesis. Since the chi-square test on the full population data also rejected the null, this high rejection rate on samples suggests that the model-based method effectively captures the association present in the population. This demonstrates that population-level inference is possible using sample data.

5.1.1 Comparison of Sample(T_s) and model-based(T_m) p-values



We plot the sorted p-values from both the sample-based test and the model-based test to compare their behavior across 1000 simulations. The green horizontal line represents the finite population p-value ($2.7819e^{-}8$). Both curves lie very close to this threshold, showing that each method strongly rejects the null hypothesis.

The model-based p-values align slightly closer to the population value, indicating that the Model-based test approach can provide a more stable approximation of the true result using only sample data.

6 Test under Stratified Sampling

So far, our analysis has relied on Simple Random Sampling Without Replacement (SRSWOR). While this provides unbiased estimates, it does not guarantee that important subgroups of the population are well represented in every sample. To address this, we now use stratified sampling, where the population is

divided into homogeneous subgroups (strata) such as education levels or regions, and samples are drawn proportionally within each stratum.

By applying the same Test of Independence under stratified sampling, we can evaluate whether this design yields more stable and reliable inference than SRSWOR.

To implement stratified sampling, we divided the population into two strata based on the binary variable binary_income_status. The total sample size was set to 200, with proportional allocation applied so that each stratum contributed to the sample in proportion to its population size. Using the sampling package in R, we selected the sample through stratified simple random sampling without replacement (SRSWOR).

Contingency Table:

```
Stratified Sample Frequency Table(f_ij):
    Income
Vote  0  1
   0 19 10
   1 83 88
```

Proportion Table:

```
Proportion Table(p_ij):
    Income
Vote     0     1
   0 0.095 0.050
   1 0.415 0.440
```

Test-statistic and p-value:

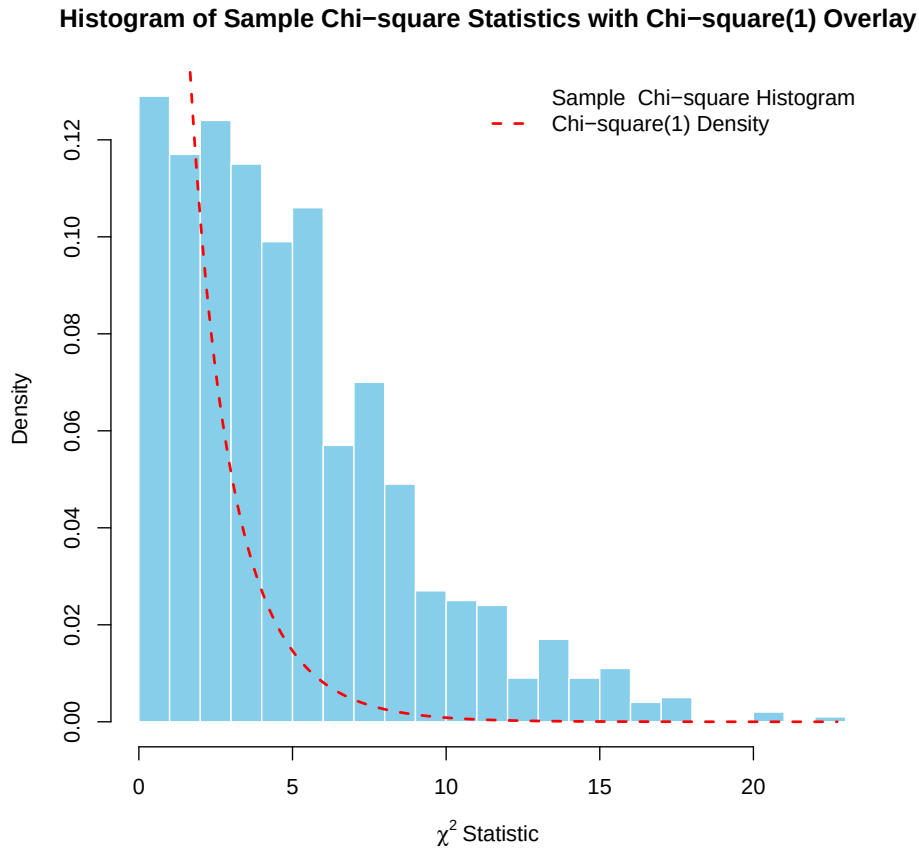
```
Chi-square statistic = 2.8604
p-value = 0.090782
```

The p-value shows clearly accept the Null hypothesis .

6.1 Simulation:

We repeated the process under stratified sampling by drawing 1000 simulated samples, computing the chi-square test for each, and storing the corresponding test statistics and p-values for further comparison.

a. We now visualize the empirical distribution of the 1000 chi-square statistics and compare it with the theoretical chi-square distribution with 1 degree of freedom:



The empirical distribution closely follows the theoretical curve, especially in the lower range, confirming that the test statistic approximately follows the chi-square distribution under repeated sampling. This supports the validity of the chi-square test in assessing independence between voting and income.

b. To estimate how often the null hypothesis of independence is rejected under repeated sampling, we examine the proportion of samples with a p-value less than 0.05. It gives us

```
Number of samples with p-value < 0.05: 545 out of 1000
Percentage of reject the null hypothesis: 54.5 %
```

Out of 1000 samples, 545 (54.5%) had p-values below 0.05, indicating rejection of the null hypothesis in over half the cases. This suggests that the chi-square test can detect the association between income and voting in many samples, but not all — likely due to sampling variability in smaller sample sizes.

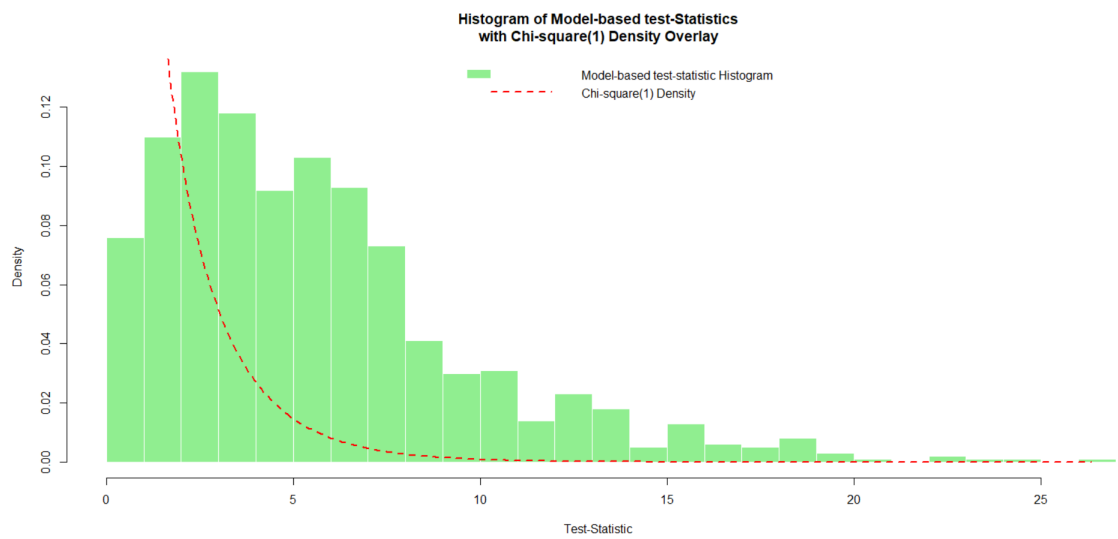
6.2 Test for Independence Using complete Auxiliary Information under Statified Sampling

Using the model-based expected frequencies, we repeated the same procedure as in the SRSWOR case, generating 1000 simulations and recording the corresponding Test- statistics and p-values for comparison.

6.3 Simulation

We repeated the process under stratified sampling by drawing 1000 simulated samples, computing the test for each, and storing the corresponding test statistics and p-values for further comparison which we do .

a. The resulting distribution of 1000 Test-statistic values is visualized in the histogram below, with the theoretical chi-square density overlaid in red for comparison. This comparison demonstrates weather the Test-statistic approximately follows the expected χ_1^2distribution under the null.



b. To estimate how often the null hypothesis of independence is rejected under repeated sampling, we examine the proportion of samples with a p-value less than 0.05. It gives us

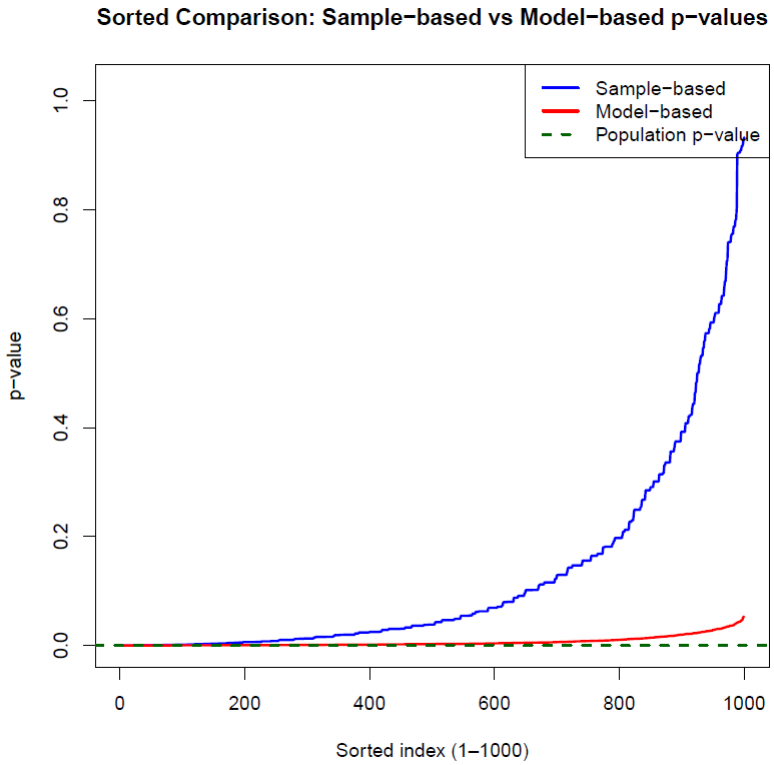
```
Number of LIMC p-values < 0.05: 997 out of 1000

Percentage of reject the null hypothesis: 99.7 %
```

Out of 1000 samples, 997 p-values were less than 0.05, leading to an 99.7% rejection rate of the null hypothesis. Since the chi-square test on the full population data also rejected the null, this high rejection rate on samples suggests that the model-based method effectively captures the association present in the

population. This demonstrates that population-level inference is possible using sample data.

Comparison of Sample (T_s) and model-based(T_M) p-values under Statified Sampling We plot the sorted p-values from both the sample-based test and the model-based test to compare their behavior across 1000 simulations. The green horizontal line represents the true population p-value ($2.7819e^{-}8$).



The plot shows that sample-based p-values are highly variable across simulations, while model-based p-values consistently align more closely with the population p-value, demonstrating the advantage of model-based inference in reducing sampling variability.

Compared to SRSWOR, Stratified sampling yields more accurate estimates of the Test of Independence.

Furthermore, when the objective is to infer population-level association from limited data, model-based tests—such as those using the bivariate probit model—yield p-values that are more stable and closer to the true population test. This demonstrates that model-based inference provides a stronger basis for drawing conclusions about the association in the population.

7 Population Generation using the Bivariate Probit Model

In this study, I use the bivariate probit model to generate a synthetic population dataset.

Model Specification Let Y_{1t}, Y_{2t} be binary random variables which take the value of 0 or 1 and are assumed to follow the joint probability distribution

In this study, we apply the Bivariate Probit model in a simulated setting to examine the relationship between two binary outcomes,Y_{1t}, Y_{2t} . The model assumes that both outcomes arise from latent continuous variables that are thresholded at zero. This framework naturally accommodates potential dependence between the two outcomes through correlated error terms.

We define

$Y_{1t} = I(U_{1t} \geq 0)$, $Y_{2t} = I(U_{2t} \geq 0)$

where the latent variables U_{1t} and U_{2t} are specified as linear functions of observed covariates plus random errors:

$$U_{1t} = \beta_0 + \beta x_t + \varepsilon_{1t}$$

$$U_{2t} = \gamma_0 + \gamma x_t + \varepsilon_{2t}$$

where $\varepsilon_{1t}$and ε_{2t} are the error terms . The error terms following a bivariate standard normal distribution with zero means, unit variances, and correlation coefficient ρ:

$$\begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right)$$

This specification implies the following marginal and joint probabilities:

$$P[Y_{1t} = 1] = P[U_{1t} \geq 0] = \phi(\beta x_t)....(i)$$

$$P[Y_{2t} = 1] = P[U_{1t} \geq 0] = \phi(\gamma x_t)....(ii)$$

$$P[Y_{1t} = 1, Y_{2t} = 1] = P[U_{1t} \geq 0, U_{2t} \geq 0] = F(\beta x_t, \gamma x_t, \rho)....(iii)$$

where ϕ is the distribution function of a standard normal variable, and F is the bivariate distribution function of standard normal variables with correlation coefficient ρ.

For our simulation study, we set the population size to T=1500 (1,2,3,...,t=1500) and generate the covariate $x_t \sim U(0, 1)$ The true parameter values are specified as β= 0.5 , γ = -0.8 and ρ =0 ,which implies independence between (Y_{1t}, Y_{2t}) under the null hypothesis. For each observation, we simulate

$U_{1t}|x_t \sim N(\beta x_t, 1)$ and $U_{2t}|x_t \sim N(\gamma x_t, 1)$

and then determine the binary outcomes according to the threshold rule:

$$Y_{1t} = \begin{cases} 1 & if \quad U_{1t} \geq 0 \\ 0 & , \quad otherwise \end{cases} \quad and\, Y_{2t} = \begin{cases} 1 & if \quad U_{2t} \geq 0 \\ 0 & , \quad otherwise \end{cases}$$

After generating the population of size 1500 using the bivariate probit model described above, we obtain a dataset containing the covariate x and the binary outcomes Y_1, Y_2 .A snapshot of the first few rows of the simulated population dataset is shown below:

```
          x Y1 Y2
1 0.2875775  1  0
2 0.7883051  1  0
3 0.4089769  1  0
4 0.8830174  1  0
5 0.9404673  1  1
6 0.0455565  0  1
```

To examine the joint distribution of the binary variables $Y_1$and Y_2in the full population of size 1500, we construct the contingency table.

This table shows the frequency counts for each combination of $Y_1 \epsilon \{0, 1\}$ and $Y_2 \epsilon \{0, 1\}$.

Contingency Table of Y_1 and Y_2

```
   Y2
Y1    0   1
  0 375 208
  1 605 312
```

Proportion Table (p_{ij})

```
   Y2
Y1          0         1
  0 0.2500000 0.1386667
  1 0.4033333 0.2080000
```

Test of Independence As we know ρ =0 , which implies independence between y_1 and $y_2$under the null hypothesis. So we calculate the chi-square statistic .

```
Chi-square Statistic: 0.3603
```

Sampling from the Population (SRSWOR Approach) Similar to the earlier analysis with the voteincome dataset, we now draw a simple random sample without replacement (SRSWOR) of size 200 from the simulated population. Using this sample, we construct the contingency table of (Y_1, Y_2) and compute the chi-square test statistic .

- Contingency Table

```
   Y2
Y1   0  1
  0 45 27
  1 78 50
```

- Proportion Table (p_{ij})

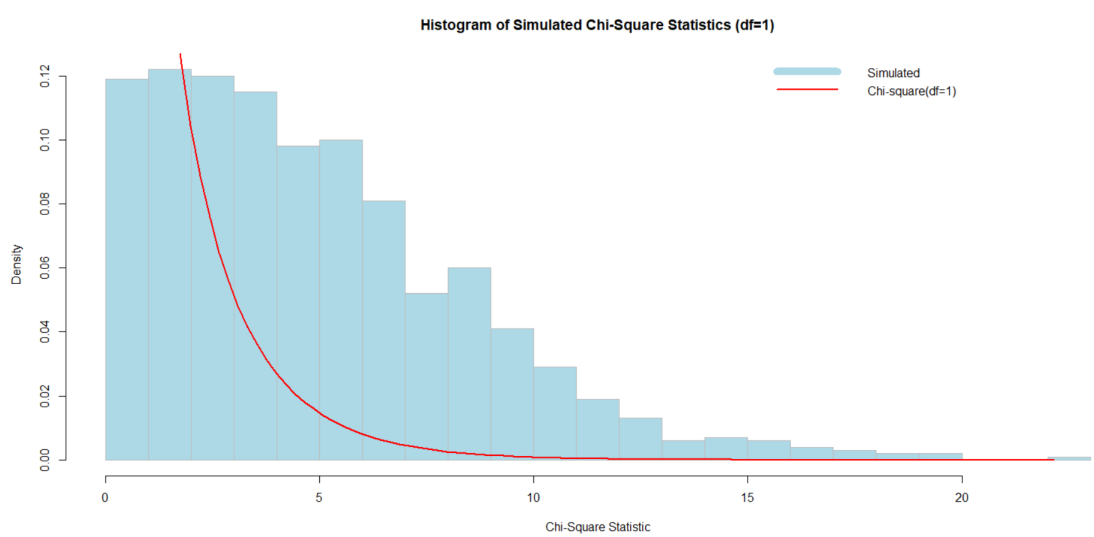
```
   Y2
Y1      0     1
  0 0.225 0.135
  1 0.390 0.250
```

- $\chi^2_1 - statistic$

```
Chi-square statistic = 0.0475
```

Simulation We repeated the process under stratified sampling by drawing 1000 simulated samples of sample size=200.

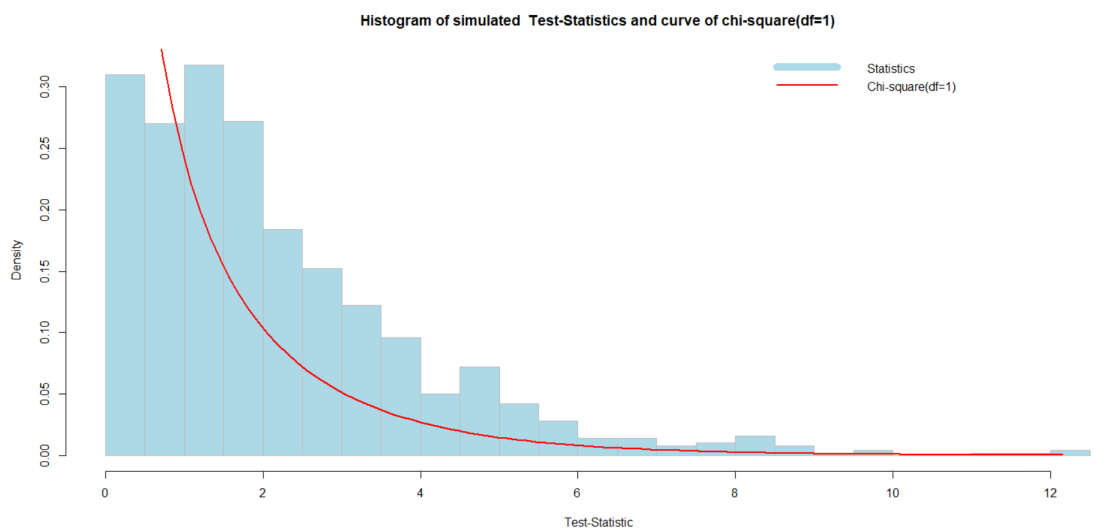
- We now visualize the empirical distribution of the 1000 chi-square statistics and compare it with the theoretical chi-square distribution with 1 degree of freedom:



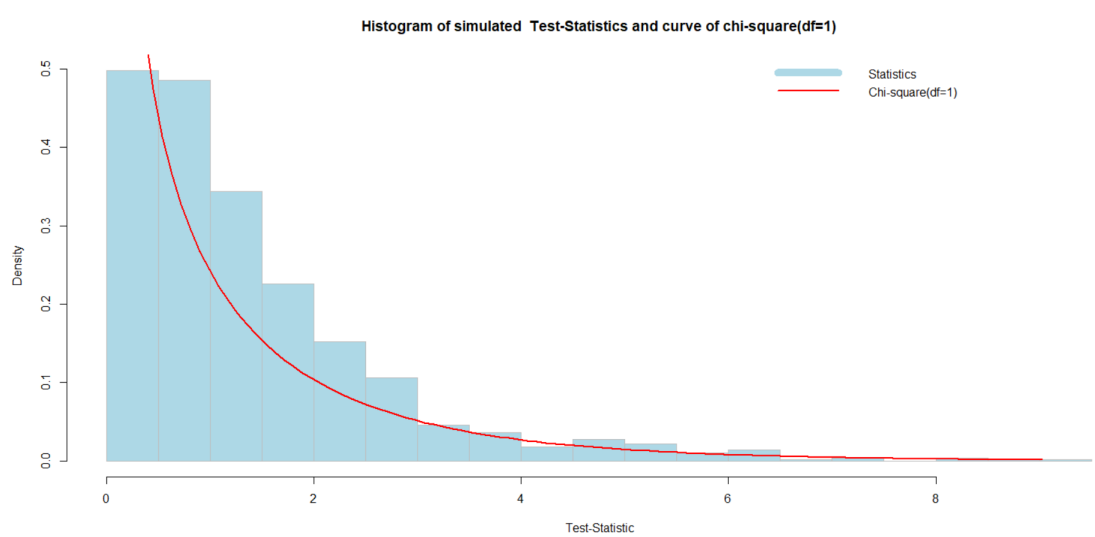
The histogram of simulated chi-square statistics aligns closely with the theoretical chi-square distribution (df=1). This confirms that the test statistic follows its expected distribution under the null hypothesis, validating the use of the chi-square test for independence.

Test for Independence Using Model-Based Expected Frequencies Using the model-based expected frequencies, we repeated the same procedure as in the voteincome dataset SRSWOR case, generating 1000 simulations and recording the corresponding Test- statistics.

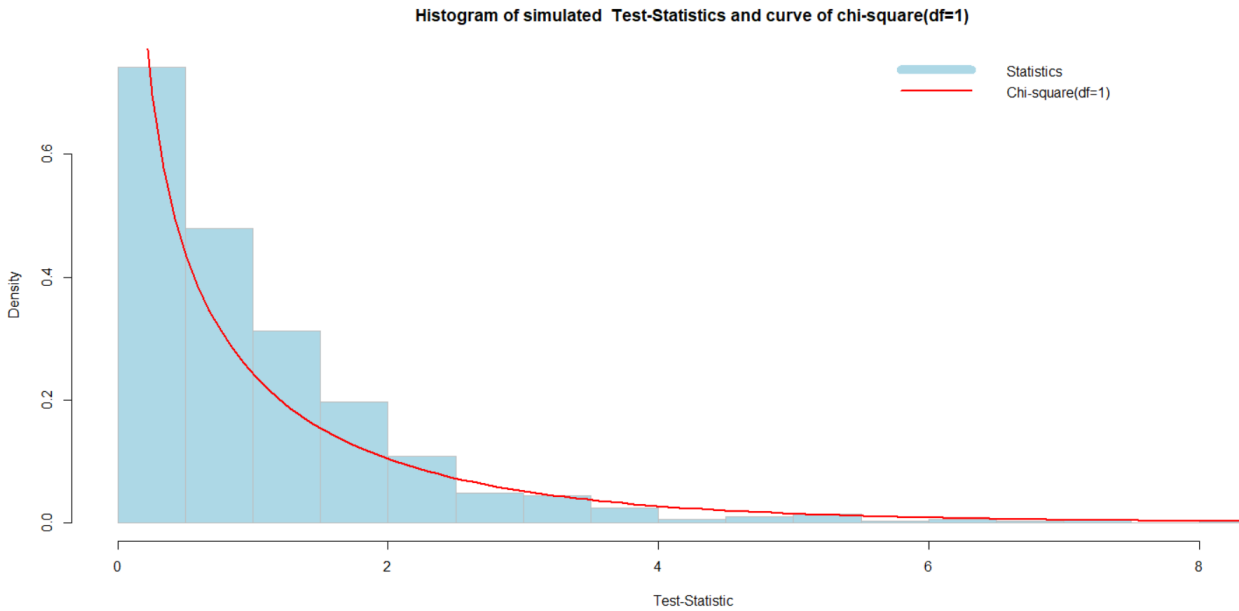
- The resulting distribution of 1000 Test-statistic values is visualized in the histogram below, with the theoretical chi-square density overlaid in red for comparison. This comparison demonstrates wheather the Test-statistic approximately follows the expected χ_1^2 distribution under the null.



The histogram roughly follows the chi-square curve.
When sample size=500



The distribution aligns more closely with the chi-square curve.
When sample size= 700



The histogram nearly overlaps the chi-square curve, showing a very close match.

Increasing sample size improves the approximation of the simulated test statistic distribution to the theoretical chi-square distribution. With larger samples, random variability diminishes, and the fit becomes more accurate.

When the variables are independent, the model-based test statistic follows a chi-square distribution with 1 degree of freedom ,so the model-based test-statistic can also be interpreted as a chi-square statistic. In the case the distribution of the test statistic depends only on the conditional distribution of $(Y_1, Y_2)|\ x$ .Whether we select a simple random sample or a stratified sample based on x the induced sampling distribution of the test statistic remains the same, because the joint law of $(Y_1, Y_2)|x$ does not change .

Since the conditional distribution of (Y_1, Y_2) is already determined by the covariate x, stratification does not change the distribution of the test statistic. Hence, we did not carry out the stratification part in this analysis

8 Conclusion

This study examined the relationship between income and voting behavior using both classical and model-based approaches. The chi-square test applied

to the full population confirmed a significant association between the variables. Through repeated sampling, we observed that the sample-based chi-square statistics approximate the theoretical chi-square distribution, validating the reliability of the test under random sampling. Furthermore, model-based estimation using the bivariate probit framework provided joint probabilities that were used to construct test-statistics for whole population with the help of sample data. The results showed that, under the null hypothesis of independence, the model-based test statistic also follows a chi-square distribution with one degree of freedom. This confirms that both classical and model-based approaches are consistent, and the model-based method offers a robust alternative for inference when only sample data are available.

9 References

1. Raymond L.Chambers and Robert G.Clark An Introduction to Model-Based Survey Sampling With Applications

2. BPM - Takeshi Amemiya Bivariate Probit Analysis :Minimum Chi-square Methods , source: Journal of the American Statistical Association ,vol.69, No. 348 (Dec., 1974)

3. χ^2test - A.M.Gun, M.K.Gupta , B.Dasgupta Fundamentals of Statistics (volume1)

10 Appendix

In this report, we developed and executed R programs to study the association between voting behavior and income status. The analysis includes constructing contingency tables, performing chi-square tests on both finite population and sampled data, simulating repeated sampling under SRSWOR and Statified, and evaluating p-values for hypothesis testing at the 5% level.

We also applied a Bivariate Probit Model to estimate joint probabilities, compute model-based chi-square statistics, and compare them with sample-based results. The scripts generate supporting visualizations, such as histograms of chi-square values and p-values.

All R codes, datasets, and outputs have been organized , representing the complete workflow of data processing, sampling, estimation, testing, and visualization.

```
install.packages("dplyr")
library(dplyr)
install.packages("devtools")
devtools::install_github("IQSS/ZeligChoice")
```

```r
devtools::install_github("IQSS/Zelig")

library(Zelig)
data(package = "Zelig")
library(ZeligChoice)
data(voteincome)

voteincome$binary_income_status <- ifelse(voteincome$income > median(voteincome$income), 1,
voteincome <- voteincome %>% select(-state, -income, -year)
voteincome <- voteincome %>%select(vote, binary_income_status, everything())
head(voteincome)
nrow(voteincome)
##########################################################
# Create contingency table of vote and Income
table_vote_Income <- table(Vote = voteincome$vote, Income= voteincome$binary_income_status)

# Print the table
print("Contingency Table of Vote and Income:\n")
print(table_vote_Income)

prop_table <- prop.table(table_vote_Income)
cat("\nProportion Table:\n")
print(round(prop_table, 3))
##############################################3
# Total sample size
N <- sum(table_vote_Income)

# Row and column marginal proportions under H0
p_i_dot <- prop.table(rowSums(table_vote_Income))  # p_{i.}
p_dot_j <- prop.table(colSums(table_vote_Income))  # p_{.j}

# Expected frequencies under H0: E_ij = n * p_{i.} * p_{.j}
expected <- outer(p_i_dot, p_dot_j) * N

# Observed frequencies
observed <- table_vote_Income

# Compute Chi-square statistic:
chi_sq_stat <- sum((observed - expected)^2 / expected)

# Print the result
cat("Chi-square statistic =", round(chi_sq_stat, 4), "\n")

p_val <- 1 - pchisq(chi_sq_stat, df = 1)
cat("p-value =", round(p_val, 6), "\n")
```

```r
##################################
# for SRSWOR
library(sampling)

# Number of observations in population
N <- nrow(voteincome)

# Draw a sample of size 200 using SRSWOR
set.seed(123)
srs_sample <- srswor(n = 200, N = N)

# Extract sampled data
sample_data <- voteincome[srs_sample == 1, ]

# Frequency table
freq_table <- table(Vote = sample_data$vote, Income = sample_data$binary_income_status)
cat("Frequency Table:\n")
print(freq_table)

# Proportion table
prop_table <- prop.table(freq_table)
cat("\nProportion Table:\n")
print(round(prop_table, 3))

# FUNCTION for chi-square statistic
chi_square <- function(freq_table) {
  # Total sample size
  n <- sum(freq_table)

  # Estimate row and column marginal proportions
  p_i_dot <- prop.table(rowSums(freq_table))  # row margins
  p_dot_j <- prop.table(colSums(freq_table))  # column margins

  # Compute expected frequencies under H0: p_ij = p_i. * p_.j
  expected <- outer(p_i_dot, p_dot_j) * n

  # Compute Chi-square statistic
  chi_sq <- sum((freq_table - expected)^2 / expected)

  return(chi_sq)
}
#############################
chi_statistic <- chi_square(freq_table)
cat("Chi-square statistic =", round(chi_statistic, 4), "\n")
```

```r
# Compute p-value (df = 1 for 2x2 table)
p_val <- 1 - pchisq(chi_statistic, df = 1)
cat("p-value =", round(p_val, 6), "\n")
#######################################
#simulation part
# --- Initialize storage
n_sim <- 1000
chi_sq_array <- numeric(n_sim)
p_val_array <- numeric(n_sim)

set.seed(123)  # For reproducibility

# --- Simulation loop
for (i in 1:n_sim) {
  srs_sample <- srswor(n = 200, N = nrow(voteincome))
  sample_data <- voteincome[srs_sample == 1, ]

  freq_table <- table(Vote = sample_data$vote, Income = sample_data$binary_income_status)

  # Check for valid 2x2 table
  if (all(dim(freq_table) == c(2, 2))) {
    chi_sq_val <- chi_square(freq_table)
    p_val <- 1 - pchisq(chi_sq_val, df = 1)

    chi_sq_array[i] <- chi_sq_val
    p_val_array[i] <- p_val
  } else {
    chi_sq_array[i] <- NA
    p_val_array[i] <- NA
  }
}
library(pbivnorm)




z.out <- zelig(cbind(vote, binary_income_status) ~ education, model = "bprobit", data = vote
summary(z.out)

x.out <- setx(z.out)
s <- sim(z.out,x = x.out)
summary(s)


# 1. Extract simulated coefficients (point estimates)
```

```r
simulated_coefs <- s$get_coef()[[1]]

# 2. Extract beta (for vote), gamma (for income), rho
beta  <- simulated_coefs[c("(Intercept):1", "education:1")]
gamma <- simulated_coefs[c("(Intercept):2", "education:2")]
Rho   <- simulated_coefs["(Intercept):3"]  # this is the correlation (rho)


# Use a subsample not in the 100-sample
non_sample <- voteincome[-sample(1:nrow(voteincome), 100), ]


X <- model.matrix(~ education, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
  eta2 <- as.numeric(X %*% gamma)


  #--Joint probabilities via bivariate normal CDF--
  P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non
all.equal(P11_non + P10_non + P01_non + P00_non, rep(1, length(P11_non)))


##################################################
# function

# Main function to compute LIMC Chi-square
limc_chisq <- function(sample_data, non_sample, freq_table, beta, gamma, Rho) {
  n <- nrow(sample_data)
  N <- n + nrow(non_sample)

  # --- Step 1: Compute expected probabilities from Bivariate Probit for non-sample ---
  X <- model.matrix(~ education, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
  eta2 <- as.numeric(X %*% gamma)

    P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non

  # --- Step 2: Estimate marginal probabilities ---
  # For Y1 (vote)
```

```r
  y1_sample <- sample_data$vote
  p1_dot <- (sum(y1_sample == 1) + sum(P11_non + P10_non)) / N

  # For Y2 (income)
  y2_sample <- sample_data$high_income
  p_dot1 <- (sum(y2_sample == 1) + sum(P11_non + P01_non)) / N

  # --- Step 3: Compute expected counts under independence ---
  expected <- matrix(0, nrow = 2, ncol = 2)
  expected[1,1] <- n * (1 - p1_dot) * (1 - p_dot1)  # P00
  expected[1,2] <- n * (1 - p1_dot) * p_dot1        # P01
  expected[2,1] <- n * p1_dot * (1 - p_dot1)        # P10
  expected[2,2] <- n * p1_dot * p_dot1              # P11

  # --- Step 4: Compute Chi-square statistic ---
  chi_sq <- sum((freq_table - expected)^2 / expected)

  return(list(
    chi_sq = chi_sq,
    p1_dot = p1_dot,
    p_dot1 = p_dot1,
    expected = expected
  ))
}



result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)

cat(" Chi-square statistic:", result$chi_sq, "\n")
cat("Estimated p1. =", round(result$p1_dot, 4), ", p.1 =", round(result$p_dot1, 4), "\n")


# Compute p-value
p_value <- 1 - pchisq(result$chi_sq, df = 1)

cat("Chi-square statistic:", round(result$chi_sq, 4), "\n")
cat("p-value:", round(p_value, 6), "\n")
########################################
# Run simulation
# --- Simulation setup
n_sim <- 1000
n_sample <- 200
limc_chi_sq_values <- numeric(n_sim)
limc_p_values <- numeric(n_sim)
```

```r
set.seed(123)  # For reproducibility
N <- nrow(voteincome)

# --- Simulation loop
for (i in 1:n_sim) {
  # Draw a simple random sample without replacement
  srs_sample <- srswor(n_sample, N)
  sample_data <- voteincome[srs_sample == 1, ]
  non_sample <- voteincome[srs_sample == 0, ]

  # Build frequency table from sample
  freq_table <- table(Vote = sample_data$vote, Income = sample_data$high_income)

  # Ensure 2x2 table before proceeding
  if (all(dim(freq_table) == c(2, 2))) {
    # Compute LIMC chi-square
    limc_result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)
    limc_chi_sq_values[i] <- limc_result$chi_sq
    limc_p_values[i] <- 1 - pchisq(limc_result$chi_sq, df = 1)
  } else {
    limc_chi_sq_values[i] <- NA
    limc_p_values[i] <- NA
  }
}
# ####################
# simulation for Statified
install.packages("sampling")
library(sampling)

# View the structure of strata variable
table(voteincome$binary_income_status)

# Create a stratum ID column (as required by `sampling` package)
voteincome$stratum <- voteincome$binary_income_status

# Total sample size
n_total <- 200

# Number of strata
strata_counts <- table(voteincome$stratum)
N_h <- as.numeric(strata_counts)
n_h <- round(n_total * N_h / sum(N_h))  # Proportional allocation

# Draw stratified sample using stratified sampling without replacement
set.seed(123)
```

```r
stratified_sample_index <- strata(voteincome, stratanames = c("stratum"),
                                  size = n_h, method = "srswor")$ID_unit

# Extract sample
stratified_sample <- voteincome[stratified_sample_index, ]

# Check size
nrow(stratified_sample)

# Display a contingency table for the sample
freq_table <- table(Vote = stratified_sample$vote, Income = stratified_sample$binary_income_
cat("Stratified Sample Frequency Table:\n")
print(freq_table)

# Proportion table
prop_table <- prop.table(freq_table)
cat("\nProportion Table:\n")
print(round(prop_table, 3))

#################################################################3
chi_square <- function(freq_table) {

  # Total sample size
  n <- sum(freq_table)

  # Estimated marginal probabilities
  p_i_dot <- rowSums(freq_table) / n
  p_dot_j <- colSums(freq_table) / n
  # Expected cell probabilities under independence
  p_ij_hat <- outer(p_i_dot, p_dot_j)
  # Expected frequencies
  expected <- p_ij_hat * n

  # Observed frequencies
  observed <- freq_table

  # Pearson Chi-square statistic
  chi_sq <- sum((observed - expected)^2 / expected)

  return(chi_sq)
}
#######################################################################
chi_statistic <- chi_square(freq_table)
cat("Chi-square statistic =", round(chi_statistic, 4), "\n")
```

```r
# Compute p-value (df = 1 for 2x2 table)
p_val <- 1 - pchisq(chi_statistic, df = 1)
cat("p-value =", round(p_val, 6), "\n")
##############################################################3
# Initialize vector to store chi-square values
chi_stat_array <- numeric(1000)

# Set seed for reproducibility
set.seed(123)

# Total sample size
n_total <- 200

# Population stratum sizes
strata_counts <- table(voteincome$binary_income_status)
N_h <- as.numeric(strata_counts)

# Proportional allocation to strata
n_h <- round(n_total * N_h / sum(N_h))

# Repeat sampling and statistic calculation 1000 times
for (i in 1:1000) {

  # Draw stratified sample
  stratified_sample_index <- strata(voteincome, stratanames = c("stratum"),
                                    size = n_h, method = "srswor")$ID_unit
  stratified_sample <- voteincome[stratified_sample_index, ]

  # Frequency table
  freq_table <- table(Vote = stratified_sample$vote, Income = stratified_sample$high_income)

  # Compute chi-square statistic
  chi_stat_array[i] <- chi_square(freq_table)
}

# Inspect first few statistics
head(chi_stat_array)

# Compute p-values from chi-square statistics (df = 1)
p_val_array <- 1 - pchisq(chi_stat_array, df = 1)

library(pbivnorm)
```

```r
z.out <- zelig(cbind(vote, high_income) ~ education, model = "bprobit", data = voteincome)
summary(z.out)

x.out <- setx(z.out)
s <- sim(z.out,x = x.out)
summary(s)


# 1. Extract simulated coefficients (point estimates)
simulated_coefs <- s$get_coef()[[1]]

# 2. Extract beta (for vote), gamma (for income), rho
beta  <- simulated_coefs[c("(Intercept):1", "education:1")]
gamma <- simulated_coefs[c("(Intercept):2", "education:2")]
Rho   <- simulated_coefs["(Intercept):3"]  # this is the correlation (rho)


# Use a subsample not in the 100-sample
non_sample <- voteincome[-sample(1:nrow(voteincome), 200), ]


X <- model.matrix(~ education, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
  eta2 <- as.numeric(X %*% gamma)


  #--Joint probabilities via bivariate normal CDF--
  P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non
all.equal(P11_non + P10_non + P01_non + P00_non, rep(1, length(P11_non)))


##################################################
# function

# Main function to compute LIMC Chi-square
limc_chisq <- function(sample_data, non_sample, freq_table, beta, gamma, Rho) {
  n <- nrow(sample_data)
  N <- n + nrow(non_sample)

  # --- Step 1: Compute expected probabilities from Bivariate Probit for non-sample ---
  X <- model.matrix(~ education, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
```

```r
  eta2 <- as.numeric(X %*% gamma)

    P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non

  # --- Step 2: Estimate marginal probabilities ---
  # For Y1 (vote)
  y1_sample <- sample_data$vote
  p1_dot <- (sum(y1_sample == 1) + sum(P11_non + P10_non)) / N

  # For Y2 (income)
  y2_sample <- sample_data$binary_income_status
  p_dot1 <- (sum(y2_sample == 1) + sum(P11_non + P01_non)) / N

  # --- Step 3: Compute expected counts under independence ---
  expected <- matrix(0, nrow = 2, ncol = 2)
  expected[1,1] <- n * (1 - p1_dot) * (1 - p_dot1)  # P00
  expected[1,2] <- n * (1 - p1_dot) * p_dot1        # P01
  expected[2,1] <- n * p1_dot * (1 - p_dot1)        # P10
  expected[2,2] <- n * p1_dot * p_dot1              # P11

  # --- Step 4: Compute Chi-square statistic ---
  chi_sq <- sum((freq_table - expected)^2 / expected)

  return(list(
    chi_sq = chi_sq,
    p1_dot = p1_dot,
    p_dot1 = p_dot1,
    expected = expected
  ))
}



result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)

cat("LIMC Chi-square statistic:", result$chi_sq, "\n")
cat("Estimated p1. =", round(result$p1_dot, 4), ", p.1 =", round(result$p_dot1, 4), "\n")


# Compute p-value
p_value <- 1 - pchisq(result$chi_sq, df = 1)
```

```r
cat("Chi-square statistic:", round(result$chi_sq, 4), "\n")
cat("p-value:", round(p_value, 6), "\n")
##############################################################
# Number of simulations
n_sim <- 1000

# Initialize arrays to store LIMC results
limc_chisq_array <- numeric(n_sim)
limc_p_value_array <- numeric(n_sim)

# Set seed for reproducibility
set.seed(123)

for (i in 1:n_sim) {

  # Step 1: Draw stratified sample
  sample_index <- strata(voteincome, stratanames = c("stratum"),
                         size = n_h, method = "srswor")$ID_unit
  sample_data <- voteincome[sample_index, ]

  # Step 2: Create freq table
  freq_table <- table(Vote = sample_data$vote, Income = sample_data$high_income)

  # Step 3: Get non-sample data
  non_sample <- voteincome[-sample_index, ]

  # Step 4: Compute LIMC chi-square and p-value
  result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)
  limc_chisq_array[i] <- result$chi_sq
  limc_p_value_array[i] <- 1 - pchisq(result$chi_sq, df = 1)
}

###################
## code for genarate independent dataset
set.seed(123)

# Step 1: Generate 1500 x values from Uniform(0,1)
n <- 1500
x <- runif(n)
# Step 2: Choose beta1 and beta2
beta1 <- 0.5
beta2 <- -0.8  # Different sign just for variety; independence doesn't require same sign

# Step 3: Generate u1 | x ~ N(beta1 * x, 1)
u1 <- rnorm(n, mean = beta1 * x, sd = 1)
```

```r
# Step 4: Generate u2 | x ~ N(beta2 * x, 1)
u2 <- rnorm(n, mean = beta2 * x, sd = 1)

# Step 5: Define binary outcomes
y1 <- ifelse(u1 >= 0, 1, 0)
y2 <- ifelse(u2 >= 0, 1, 0)

# Step 6: Estimate probabilities
p_y1_1 <- mean(y1 == 1)
p_y2_1 <- mean(y2 == 1)
p_y1_1_y2_1 <- mean((y1 == 1) & (y2 == 1))

# Step 7: Compare product vs joint
cat("P(Y1 = 1):", round(p_y1_1, 4), "\n")
cat("P(Y2 = 1):", round(p_y2_1, 4), "\n")
cat("P(Y1 = 1, Y2 = 1):", round(p_y1_1_y2_1, 4), "\n")
cat("P(Y1 = 1) * P(Y2 = 1):", round(p_y1_1 * p_y2_1, 4), "\n")
#######################################################
set.seed(123)
# Create a data frame of the population
population_data <- data.frame(x = x, y1 = y1, y2 = y2)
head(population_data)
#####################################
# Create a 2x2 contingency table of y1 and y2
contingency_table <- table(Y1 = y1, Y2 = y2)

# Print the contingency table
cat("Contingency Table of Y1 and Y2:\n")
print(contingency_table)

##########################################
chi_test <- chisq.test(contingency_table)
print(chi_test)
#################################################
# Install if not already installed
install.packages("sampling")
library(sampling)

# Number of observations in population
N <- nrow(population_data)

# Draw a sample of size 200 using SRSWOR
set.seed(123)
srs_sample <- srswor(n = 200, N = N)
```

```r
# Extract sampled data
sample_data <- population_data[srs_sample == 1, ]
head(sample_data)
# Frequency table
freq_table <- table(Y1= sample_data$y1, Y2= sample_data$y2)
cat("Frequency Table:\n")
print(freq_table)
# Proportion table
prop_table <- prop.table(freq_table)
cat("\nProportion Table:\n")
print(round(prop_table, 3))

##################################3
chi_square <- function(freq_table) {

  # Total sample size
  n <- sum(freq_table)

  # Estimated marginal probabilities
  p_i_dot <- rowSums(freq_table) / n
  p_dot_j <- colSums(freq_table) / n
  # Expected cell probabilities under independence
  p_ij_hat <- outer(p_i_dot, p_dot_j)
  # Expected frequencies
  expected <- p_ij_hat * n

  # Observed frequencies
  observed <- freq_table

  # Pearson Chi-square statistic
  chi_sq <- sum((observed - expected)^2 / expected)

  return(chi_sq)
}
#####################################
chi_statistic <- chi_square(freq_table)
cat("Chi-square statistic =", round(chi_statistic, 4), "\n")

# Compute p-value (df = 1 for 2x2 table)
p_val <- 1 - pchisq(chi_statistic, df = 1)
cat("p-value =", round(p_val, 6), "\n")
#######################################################
# --- Initialize storage arrays
n_sim <- 1000
sample_size <- 200
```

```r
chi_sq_array <- numeric(n_sim)
p_val_array <- numeric(n_sim)

set.seed(123)
# --- Simulation loop
for (i in 1:n_sim) {
  # Draw SRSWOR sample of size 200
  srs_sample <- srswor(n = sample_size, N = nrow( population_data))
  sample_data <- population_data[srs_sample == 1, ]

  # Build contingency table
  freq_table <- table(Y1= sample_data$y1, Y2= sample_data$y2)

  # Ensure it's a 2x2 table
  if (all(dim(freq_table) == c(2, 2))) {
    chi_sq_val <- chi_square(freq_table)
    p_val <- 1 - pchisq(chi_sq_val, df = 1)

    chi_sq_array[i] <- chi_sq_val
    p_val_array[i] <- p_val
  } else {
    chi_sq_array[i] <- NA
    p_val_array[i] <- NA
  }
}
#########################
##############################################################################33
library(pbivnorm)
library(Zelig)
library(ZeligChoice)

# Use your existing `population_data` with x, y1, y2
head(population_data)
z.out <- zelig(cbind(y1, y2) ~ x, model = "bprobit", data = population_data)
summary(z.out)

x.out <- setx(z.out)
s <- sim(z.out,x = x.out)
summary(s)


# 1. Extract simulated coefficients (point estimates)
simulated_coefs <- s$get_coef()[[1]]

# 2. Extract beta (for vote), gamma (for income), rho
```

```r
beta  <- simulated_coefs[c("(Intercept):1", "x:1")]
gamma <- simulated_coefs[c("(Intercept):2", "x:2")]
Rho   <- simulated_coefs["(Intercept):3"]  # this is the correlation (rho)


# Use a subsample not in the 100-sample
non_sample <- population_data[-sample(1:nrow(population_data), 200), ]
X <- model.matrix(~ x, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
  eta2 <- as.numeric(X %*% gamma)


  #--Joint probabilities via bivariate normal CDF--
  P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non
all.equal(P11_non + P10_non + P01_non + P00_non, rep(1, length(P11_non)))
##################################################
# function

# Main function to compute LIMC Chi-square
limc_chisq <- function(sample_data, non_sample, freq_table, beta, gamma, Rho) {
  n <- nrow(sample_data)
  N <- n + nrow(non_sample)

  # --- Step 1: Compute expected probabilities from Bivariate Probit for non-sample ---
  X <- model.matrix(~ x, data = non_sample)
  eta1 <- as.numeric(X %*% beta)
  eta2 <- as.numeric(X %*% gamma)

    P11_non <- mapply(pbivnorm, eta1, eta2, MoreArgs = list(rho = Rho))
  P10_non <- pnorm(eta1) - P11_non
  P01_non <- pnorm(eta2) - P11_non
  P00_non <- 1 - P11_non - P10_non - P01_non

  # --- Step 2: Estimate marginal probabilities ---
  # For Y1
  y1_sample <- sample_data$y1
  p1_dot <- (sum(y1_sample == 1) + sum(P11_non + P10_non)) / N

  # For Y2
  y2_sample <- sample_data$y2
  p_dot1 <- (sum(y2_sample == 1) + sum(P11_non + P01_non)) / N
```

```r
  # --- Step 3: Compute expected counts under independence ---
  expected <- matrix(0, nrow = 2, ncol = 2)
  expected[1,1] <- n * (1 - p1_dot) * (1 - p_dot1)  # P00
  expected[1,2] <- n * (1 - p1_dot) * p_dot1        # P01
  expected[2,1] <- n * p1_dot * (1 - p_dot1)        # P10
  expected[2,2] <- n * p1_dot * p_dot1              # P11

  # --- Step 4: Compute Chi-square statistic ---
  chi_sq <- sum((freq_table - expected)^2 / expected)

  return(list(
    chi_sq = chi_sq,
    p1_dot = p1_dot,
    p_dot1 = p_dot1,
    expected = expected
  ))
}



result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)

cat("Estimated Chi-square statistic:", result$chi_sq, "\n")
cat("Estimated p1. =", round(result$p1_dot, 4), ", p.1 =", round(result$p_dot1, 4), "\n")


# Compute p-value
p_value <- 1 - pchisq(result$chi_sq, df = 1)

cat("Chi-square statistic:", round(result$chi_sq, 4), "\n")
cat("p-value:", round(p_value, 6), "\n")
#########################################
# Run simulation
# --- Simulation setup
n_sim <- 1000
n_sample <- 700
limc_chi_sq_values <- numeric(n_sim)
limc_p_values <- numeric(n_sim)

set.seed(123)
N <- nrow(population_data)

# --- Simulation loop
for (i in 1:n_sim) {
  # Draw a simple random sample without replacement
```

```r
  srs_sample <- srswor(n_sample, N)
  sample_data <- population_data[srs_sample == 1, ]
  non_sample <- population_data[srs_sample == 0, ]

  # Build frequency table from sample
  freq_table <- table(Y1= sample_data$y1, Y2= sample_data$y2)

  # Ensure 2x2 table before proceeding
  if (all(dim(freq_table) == c(2, 2))) {
    # Compute LIMC chi-square
    limc_result <- limc_chisq(sample_data, non_sample, freq_table, beta, gamma, Rho)
    limc_chi_sq_values[i] <- limc_result$chi_sq
    limc_p_values[i] <- 1 - pchisq(limc_result$chi_sq, df = 1)
  } else {
    limc_chi_sq_values[i] <- NA
    limc_p_values[i] <- NA
  }
}
```